ABASYN UNIVERSITY, ISLAMABAD CAMPUS.

BS -
(COMPUTER SCEINCES **)**

OPERATING SYSTEM

REG. NO :- 9008

NAME :- ASIM SULTAN

DEPARTMENT OF COMPUTING

LECTURER :- SADAF TANVIR

Assignment No. 2

Operating Systems: Process Synchronization and Memory Management

Part A: Process Synchronization

Q1. Describe two kernel data structures in which race conditions are possible. Be sure to include a description of how a race condition can occur.

A race condition occurs when multiple threads or processes access and manipulate shared data concurrently, and the final outcome depends on the exact order or timing of execution. Two kernel data structures highly vulnerable to race conditions are:

1. The Process Table / PID Bitmap: The kernel tracks active processes and allocates unique Process IDs (PIDs) using bitmaps or linked arrays. If two distinct processes call fork() concurrently on separate CPU cores, both cores execute kernel code to find the next available bit. If Core 1 reads the bitmap and identifies PID 502 as free, but before it completes the allocation write operation, Core 2 reads the identical bitmap location, it will also register PID 502 as free. Both cores will then assign PID 502 to different processes, leading to critical corruption of the process table.

2. Memory Free Lists (Linked Lists for Allocation): The kernel manages pools of unallocated memory using linked list structures. If two kernel threads attempt to allocate memory blocks concurrently, they will both read the head pointer of the free list. If Thread A grabs the head block and updates the pointer to the next block, but Thread B had already read the original head pointer right before that update, Thread B will attempt to claim the exact same memory address. This results in memory collision and immediate kernel panic.

Q2. Assume that a system has multiple processing cores. For each of the following scenarios, describe which is a better locking mechanism: a spinlock, or a mutex lock where waiting processes sleep while waiting for the lock to become available.

- **The lock is to be held for a short duration.**
- **The lock is to be held for a long duration.**
- **A thread may be put to sleep while holding the lock.**

• **The lock is to be held for a short duration: Spinlock is better.** Putting a process to sleep and waking it up requires a heavy context switch. If the lock duration is brief, it is much more efficient for a CPU core to busy-wait ('spin') in a tight loop for a few nanoseconds than to endure the structural overhead of a context switch.

• **The lock is to be held for a long duration: Mutex lock is better.** Spinning consumes 100% of a CPU core's execution cycles. If a lock is held for a long duration, spinning wastes a massive amount of processing power. Putting the thread to sleep frees up the core to run other productive threads.

• **A thread may be put to sleep while holding the lock: Mutex lock is required.** If a thread takes a spinlock and then goes to sleep, any other thread attempting to acquire the lock on that core will spin indefinitely. Since the sleeping thread cannot wake up to release it without CPU attention, this causes a catastrophic deadlock.

Q3. Assume that a context switch takes T time. Suggest an upper bound (in terms of T) for holding a spinlock. If the spinlock is held for any longer, a mutex lock (where waiting threads are put to sleep) is a better alternative.

The theoretical upper bound for holding a spinlock is **2T**.

Why: When a thread fails to acquire a mutex lock, the system incurs a context switch overhead of T to put the thread to sleep. Once the lock is released, another context switch overhead of T is required to wake the thread and resume execution. The total context switch overhead is therefore T + T = 2T. If a thread spins for longer than 2T, the CPU cycles wasted during busy-waiting exceed the total time required to perform the two context switches. Thus, for any duration longer than 2T, a mutex lock is more efficient.

Q4. A multithreaded web server wishes to keep track of the number of requests it services (known as hits). Consider the two following strategies to prevent a race condition on the variable hits...
Strategy 1: Mutex Lock
Strategy 2: Atomic Integer
Explain which of these two strategies is more efficient.

The second strategy (Atomic Integer) is significantly more efficient.

Why:
1. Mutex Strategy Overhead: A standard mutex requires invoking kernel primitives. When high traffic occurs and multiple threads attempt to increment 'hits' simultaneously, threads will be blocked, appended to a wait queue, and context-switched out. This incurs massive operating

system overhead.

2. Atomic Integer Efficiency: Atomic operations are implemented directly at the hardware level using specialized CPU instructions (such as 'LOCK XADD' on x86 architectures). They update the variable in a single, uninterruptible clock cycle without software locking, queue management, or triggering context switches, making them vastly superior for simple counters.

Q5. Consider the code example for allocating and releasing processes...

a. Identify the race condition(s).

b. Indicate where the locking needs to be placed to prevent the race condition(s).

c. Could we replace the integer variable with an atomic integer to prevent the race condition(s)?

a. Race Condition: The race condition occurs on the shared variable 'number_of_processes'. The increment operation (number_of_processes++) compiles into three distinct machine-level operations: read from memory, modify register, and write back to memory. Concurrently executing threads can interleave during these cycles, resulting in inaccurate process tracking.

b. Lock Placement: The lock must protect the condition check and the modification completely:

```
mutex.acquire();
if (number_of_processes >= MAX_PROCESSES) {
   mutex.release(); // Crucial to prevent permanent deadlock
   return -1;
}
number_of_processes++;
mutex.release();
```

c. Replacement with Atomic Integer: No, simply using an atomic integer is insufficient. While an atomic increment ensures the count itself changes safely, it does not bind the conditional check ('if number_of_processes >= MAX_PROCESSES') and the subsequent increment into a single atomic transaction. A thread could check the condition, find it valid, and get preempted before executing the increment. Another thread could do the same, pushing the count beyond MAX_PROCESSES. A mutex is necessary to enforce mutual exclusion over the entire check-and-modify sequence.

Part B: Memory Management Strategies (MFT and MVT)

Q6. MFT Allocation and Fragmentation Analysis

Partitions: P1 (100 KB), P2 (200 KB), P3 (300 KB), P4 (400 KB)
Job Arrivals (First-Fit): J1 (250 KB), J2 (90 KB), J3 (350 KB), J4 (120 KB), J5 (180 KB)

Job	Size (KB)	Assigned Partition	Internal Fragmentation (KB)
J1	250	P3 (300 KB)	50 KB
J2	90	P1 (100 KB)	10 KB
J3	350	P4 (400 KB)	50 KB
J4	120	P2 (200 KB)	80 KB
J5	180	Must Wait (FIFO)	-

b. Total Internal Fragmentation Remaining: 50 KB + 10 KB + 50 KB + 80 KB = **190 KB**.

c. Unrunnable Job: Any job requiring more than 400 KB can never run under this system because the largest static partition is fixed at 400 KB. This is a fundamental limitation of MFT. To fix this, the system design should switch to *MVT (Multiprogramming with a Variable Number of Tasks)* to dynamically resize boundaries, or implement *Paging / Virtual Memory*, allowing a process to occupy non-contiguous blocks.

Q7. MVT Allocation Memory State Trace

Initial Memory: 640 KB Single Free Hole

Step / Event	Action Description	Memory Layout State
Initial	System Booted	[Hole: 640 KB]
Event 1	P1 requests 250 KB	[P1: 250 KB] -> [Hole: 390 KB]
Event 2	P2 requests 150 KB	[P1: 250 KB] -> [P2: 150 KB] -> [Hole: 240 KB]
Event 3	P3 requests 100 KB	[P1: 250 KB] -> [P2: 150 KB] -> [P3: 100 KB] -> [Hole: 140 KB]
Event 4	P1 releases 250 KB	[Hole A: 250 KB] -> [P2: 150 KB] -> [P3: 100 KB] -> [Hole B: 140 KB]
Event 5	P4 requests 200 KB	[P4: 200 KB] -> [Hole A: 50 KB] -> [P2: 150 KB] -> [P3: 100 KB] -> [Hole B: 140 KB]
Event 6	P2 releases 150 KB	[P4: 200 KB] -> [Hole A: 50 KB] -> [Hole C: 150 KB] -> [P3: 100 KB] -> [Hole B: 140 KB]

b. Final Layout Metrics:

• Separate Holes: 3 holes exist (Hole A: 50 KB, Hole C: 150 KB, Hole B: 140 KB).

• Total External Fragmentation: 50 KB + 150 KB + 140 KB = 340 KB.

• Can a 300 KB request be satisfied? No. Under contiguous variable-partition allocation, a process must be loaded into a single continuous address block. No single block is large enough to satisfy 300 KB.

c. Compaction Analysis:

• Minimum Data to Move: 100 KB. P4 (200 KB) is already at the beginning of memory. By shifting P3 (100 KB) to sit flush against P4, all free blocks consolidate at the end.

• Operational Drawback: Compaction requires extreme CPU and memory bus overhead. The OS must suspend all active processes, duplicate and move data across memory locations, and completely recalculate all internal base memory pointers.

Q8. Dynamic Workload Analysis: MFT vs. MVT

Total System Memory: 800 KB
Job Workload Sequence: 120 KB, 350 KB, 90 KB, 200 KB, 400 KB, 60 KB

a. **MFT Configuration (Five 160 KB Fixed Partitions):**

• Limits: Any job exceeding 160 KB can never run. Therefore, the 350 KB, 200 KB, and 400 KB jobs can never be loaded.

• Max Degree of Multiprogramming: 3 (Jobs 120 KB, 90 KB, and 60 KB can all sit simultaneously in three partitions).

b. MVT Configuration (Best-Fit, No Compaction):

• Allocation Trace: 120 KB loads (680 KB left) -> 350 KB loads (330 KB left) -> 90 KB loads (240 KB left) -> 200 KB loads (40 KB left).

• Max Degree of Multiprogramming: 4 processes concurrent.

• Memory Utilization: Total used space is 120 + 350 + 90 + 200 = 760 KB. Utilization = (760 / 800) * 100 = **95%**.

c. Core Design Trade-off:

MVT significantly outperforms MFT for this workload by allowing a higher degree of multiprogramming (4 vs 3) and substantially better memory utilization (95% vs 33.75%). However, MVT introduces a higher operating system overhead due to the necessity of running dynamic allocation algorithms (Best-Fit) and resolving external fragmentation, whereas MFT requires zero structural calculations at runtime.

Q9. Allocation Simulator: Best-Fit vs. Worst-Fit

Initial Holes: H1 (20 KB), H2 (15 KB), H3 (18 KB), H4 (8 KB), H5 (6 KB)
Requests: R1 = 12 KB, R2 = 10 KB, R3 = 9 KB, R4 = 15 KB

Request	Best-Fit Allocation Trace	Worst-Fit Allocation Trace
R1 (12 KB)	Allocated in H2 (15 KB). Leftovers: H2 becomes 3 KB.	Allocated in H1 (20 KB). Leftovers: H1 becomes 8 KB.
R2 (10 KB)	Allocated in H3 (18 KB). Leftovers: H3 becomes 8 KB.	Allocated in H3 (18 KB). Leftovers: H3 becomes 8 KB.
R3 (9 KB)	Allocated in H1 (20 KB). Leftovers: H1 becomes 11 KB.	Allocated in H2 (15 KB). Leftovers: H2 becomes 6 KB.
R4 (15 KB)	Fails to allocate. No hole >= 15 KB. R4 must wait.	Fails to allocate. No hole >= 15 KB. R4 must wait.

c. Final Analysis Metrics:

• Best-Fit Remaining Ext. Frag: 11 + 3 + 8 + 8 + 6 = 36 KB. (R4 Unsatisfied).
• Worst-Fit Remaining Ext. Frag: 8 + 6 + 8 + 8 + 6 = 36 KB. (R4 Unsatisfied).

d. Long-Term Worst-Fit Outperformance: Worst-Fit deliberately leaves behind the largest possible remnants. Best-Fit creates tiny, unusable fragments (like the 3 KB sliver in H2). Over a long timeline, if a system processes mid-sized incoming requests, Worst-Fit's remnants remain large enough to hold data, while Best-Fit chokes the system because all its free memory becomes divided into fragments too tiny to hold any process.

Q10. The 35% Free Memory Fragmentation Problem

a. The Phenomenon: External Fragmentation. Although 35% of total system memory is reported as free, it is scattered across non-contiguous blocks. Because MVT mandates contiguous allocation, an incoming request will be rejected if its size exceeds that of the single largest continuous free hole available.

b. Proposed Solutions:

1. Contiguous Approach (Compaction): Introduce a compaction algorithm to shift active processes to one end of memory, consolidating all disjoint slots into a single contiguous pool.

2. Non-Contiguous Approach (Paging): Migrate the architecture to Paged Memory Management. By segmenting process memory into fixed pages and physical memory into frames, chunks can be mapped anywhere, entirely eliminating external fragmentation.

c. Operational Costs of Compaction:

Compaction forces a 'stop-the-world' latency penalty. All active execution threads must be suspended to update absolute address registers safely. This cost becomes completely unacceptable under two main workload conditions: **Real-Time Systems** where strict transactional deadlines are mandatory, and **High Direct Memory Access (DMA) Workloads** where continuous background hardware transfers prevent memory blocks from being moved safely without causing data loss.